

CODEUP

Quick How-To Guide

For learners and instructors

CodeUp is a browser-based Python learning environment for beginners. You can write and run Python, inspect output, ask questions about code, understand errors, use guided learning tools, and work inside an instructor-managed classroom.

Open CodeUp: <https://code-up-fmqr.onrender.com/ide>

Start in five minutes

1. Open CodeUp and wait for the workspace to finish loading.
2. Move to the Code editor and type a short Python program.

```
name = "Learner"  
print("Hello", name)
```

3. Press Ctrl+Enter or use Run.
4. Check Program output. Change the name, run again, and confirm the output changes.
5. Use the command box to ask something such as "explain this code" or "give me a code map".
6. If your instructor has given you a class code, open Classroom and join with your name and code.

Good to know: Voice control is optional. Every core learner task can be completed with the keyboard and normal on-screen controls.

1. The learner workspace

Most day-to-day work happens in the main IDE. You do not need to learn every feature before writing your first program.

Area	What it is for
Code editor	Write and edit Python.
Run	Execute the current program.
Program output	See printed output and runtime errors.
Command box	Ask CodeUp to explain, navigate, edit, debug, or guide you.
Voice	Say the same kinds of commands when microphone control is enabled.
Learning tools	Open optional tools such as code maps, stepping, snippets, projects, lessons, and Audio Blocks.
Classroom	Join a class, open assignments, ask for help, and submit work.

Essential keyboard shortcuts

Shortcut	Action
Ctrl+Enter	Run the current Python program.
Tab / Shift+Tab	Move to the next or previous control. In the code editor this leaves the editor (the default).
Ctrl+] / Ctrl+[	Indent or outdent the current line of code.
Ctrl+M	Move focus out of the code editor.
Ctrl+Shift+M	Toggle microphone voice control.
Ctrl+Shift+P	Open the command palette.
Alt+Shift+K	Open the full keyboard shortcuts list.
Alt+Shift+O	Open settings.
Escape	Cancel an active program-input prompt or stop CodeUp speech when applicable.

Write and run Python

- Type Python directly into the editor.
- Use normal Python indentation. By default, Tab moves to the next control and Shift+Tab moves to the previous control, so keyboard users can always leave the code editor. Use Ctrl+] to indent a line and Ctrl+[to outdent it. If you prefer Tab to indent code, turn off "Tab Leaves Editor" in CodeUp's display and accessibility settings.
- Press Ctrl+Enter or Run.
- Read Program output after each run.
- If the program fails, leave the error visible and ask CodeUp to explain it before changing everything at once.

2. Program input, output, and errors

Using input()

When a running Python program reaches `input()`, CodeUp can ask for the value while the program is waiting.

```
name = input("What is your name? ")
age = input("How old are you? ")
print(name, age)
```

- When the input prompt appears, type the answer into the input field.
- Press Enter in the field, or activate Submit with Enter or Space.
- Use Escape or the Cancel button to cancel the active prompt.
- For several `input()` calls, answer each prompt in order. CodeUp continues to the next prompt automatically.
- Answers such as run, stop, yes, no, hello, 16, Alex, or one are treated as program input while the program is waiting for input.

Alternative input commands: You can also use short commands such as "read input prompt", "use Alexis", "use 16", "read input values", "clear input values", and "cancel input".

Useful output commands

Command	What it does
read output	Read the stored output.
read last output	Replay the most recent output.
how many lines of output	Report output length.
first line of output	Read the first output line.
last line of output	Read the last output line.
show run history	Summarize recent runs.

When something breaks

A simple debugging loop works well: run the code, inspect the actual error, find where it happened, understand the cause, make one change, then run again.

Try asking	Purpose
explain error	Explain the recent error.
where did it crash	Identify the crash location.
what caused this	Explain the cause using the current error.
what value caused this	Show the relevant value when CodeUp has evidence for it.
what should I test next	Suggest the next debugging check.
fix with explanation	Propose a repair with an explanation.

3. Asking CodeUp for help

The command box accepts natural-language instructions. Voice commands use the same idea: say what you would otherwise type.

Understand your code

Example command	Use it for
explain this code / walk me through this code	Understand what the program does.
analyze code	Review the current code.
analyze deeper	Request a more detailed explanation.
give me a code map	See the program structure.
where am I	Get your current code context.
read around me	Read nearby lines.
why is this line indented	Explain the current line's nesting.
what contains this line	Read the enclosing block chain.
what variables exist	List variables found in the program.
show program state	Trace and summarize program state.
step through this	Walk through execution step by step.
find total	Find whole-word uses of a name and explain their roles.

Create or edit code

Example command	Example result
make a calculator	Generate a beginner starter program.
insert print Finished	Add a print statement.
add an if statement for score > 50	Add a conditional.
insert a for loop	Add a loop.
rename variable total to score_total	Rename a symbol.
comment this line	Comment the current line.
simplify this code	Suggest a simpler version.

Review changes before keeping them

For meaningful edits, use the change-review workflow instead of accepting changes blindly:

- what changed
- read before and after
- explain this change
- is this risky
- apply this change
- undo last change
- compare before and after

4. Optional learning and project tools

You can ignore these until you need them. They are extra learning tools, not requirements for running a normal Python program.

Tool	What it helps with
Audio Code Map	Understand functions, classes, loops, blocks, and where you are in the program.
Step Narration	Follow execution one step at a time.
Variable Watch	Track selected variables and program state.
Audio Blocks	Build beginner logic using numbered blocks, then compile or transfer it to Python.
Tutorial	Learn the interface through a guided onboarding flow.
Programming Literacy / Tutor Mode	Practice concepts, request progressive hints, and check understanding.
Python Foundations	Work through 12 modules: printing, variables, input, data types, conditions, loops, lists, dictionaries, functions, debugging, and a mini project.
Guided projects	Complete larger tasks with checkpoints.
Snippets	Save and reuse small pieces of code.
Multi-file projects	Create, open, run, and export projects with more than one file.

A few useful project commands

- read project files
- open helpers.py
- create file data.py
- run main.py
- project map
- export this project
- export for VS Code

Beginner recommendation: Start with the normal Python editor, Run, Program output, and the command box. Add the other tools when they solve a specific learning problem.

5. Classroom guide for learners

Join a class

1. Open Classroom in the main IDE.
2. Enter the class code provided by your instructor.
3. Enter your learner name and select Join.
4. You can also type a command such as "Join ABC123".

Assignments

- Open a published assignment from the classroom area.
- Read the title and instructions before editing.
- Write and run your code in the normal IDE.
- Use only the kinds of assistance allowed for that assignment.
- Submit when the code represents your intended answer.
- Use Back to IDE or Back to CodeUp to return to the normal workspace.

Useful classroom commands

Command	Action
open my assignments	Show your assignments.
what am I supposed to do	Repeat the current assignment goal.
what can AI help me with	Show what assistance is allowed.
submit / turn this in	Submit the current assignment.
I need help	Create an instructor help request.
is my instructor helping	Check the help-request state.
cancel help	Cancel an open help request.
what class am I in	Show the current classroom.
leave class	Leave/switch the learner identity on this browser.

Assignment permissions: An instructor can limit generation, fixes, explanations, hints, error help, concept questions, code maps, step narration, or variable-state tools depending on the activity.

6. Instructor quickstart

Instructor sign-in: <https://code-up-fmgr.onrender.com/classroom/instructor/login>

Set up a class

1. Create an instructor account and sign in.
2. Create a cohort with a clear name, for example "Python Beginners - Cohort A".
3. Share the cohort join code with the intended learners.
4. Create an assignment with a title, instructions, optional starter code, dates if needed, expected concepts, and assistance policy.
5. Review the assignment, then publish it.
6. Keep the cohort dashboard open during class to monitor learners, submissions, and help requests.

What the instructor dashboard is for

Area	Use
Learners	See enrolled learners, recent activity, status, and progress.
Assignments	Create, publish, duplicate, schedule, and review work.
Help queue	See waiting learners, mark a request as helping, and resolve it.
Submissions	Inspect saved/submitted code and run information.
Assistance policy	Choose what kinds of CodeUp help are allowed for an assignment.
Reports	Review cohort progress and export classroom data when needed.

Optional instructor tools

- Create custom lessons for a specific concept.
- Create guided projects with deterministic checkpoints.
- Archive completed cohorts and reactivate them later if needed.
- Use learner detail pages when you need a closer look at progress or submissions.

Before the first session

- Test instructor sign-in and create a sample cohort.
- Join once from a learner browser using the class code.
- Publish and open one sample assignment.
- Test a help request and one submission.
- Confirm Run, output, input(), and the command box work on the learner devices you will use.

7. Troubleshooting and useful links

Problem	What to try
Voice does not respond	Type the same command instead. If you want voice, check browser microphone permission and make sure voice listening is active.
A command is misunderstood	Use a shorter phrase or type the command. The Help / all commands surfaces show supported actions.
Program is waiting for input	Answer the visible input prompt, or use a short value command such as "use Alexis" or "use 16".
Program output is very long	Ask "how many lines of output", "first line of output", or "last line of output", then use "read output" when needed.
Assignment is not visible	Ask the instructor to confirm that it is published and currently available, then refresh the classroom view.
Code produces an error	Run once, keep the error visible, and try "explain error" or "where did it crash" before requesting a full fix.
A project/file cannot be found	Use "read project files" or "project map" to confirm the active project and file names.

Useful links

- Learner IDE: <https://code-up-fmqr.onrender.com/ide>
- Instructor sign-in: <https://code-up-fmqr.onrender.com/classroom/instructor/login>
- Instructor dashboard: <https://code-up-fmqr.onrender.com/classroom/instructor>
- Learner classroom home: <https://code-up-fmqr.onrender.com/classroom/learner>
- Learner join page: <https://code-up-fmqr.onrender.com/classroom/join>

For a first class: Keep the workflow simple: join the class, write code, Run, inspect output, ask for help when needed, and submit. The advanced tools can be introduced gradually.

End of quick guide

CodeUp is designed to make beginner Python practice easier to navigate, explain, debug, and teach. You do not need to use every feature at once.